Institut National des Postes et Télécommunications

Filière : Smart ICT

Rapport :

Projet de Communication VoIP

Sujet : Implémentation d'une solution VoIP basée sur SIP/RTP

Encadré par :
Mr. Abdelali CHAOUB

Travail effectué par :
AHOULIMI BIDENIMM
SIDIKI ABOUBAKAR
EL KARFI SOUFIA
LAZAAR EL MEHDI

Avril 2026

Table des matières

1 Introduction **3**

2 Historique : Internet et WebRTC **4**
2.1 Évolution d'Internet 4
2.2 De la VoIP classique aux architectures temps réel modernes 4
2.3 Historique de WebRTC 5

3 Explication des protocoles SIP et RTP **5**
3.1 SIP : signalisation des sessions 5
3.2 Structure des messages SIP et champs critiques 6
3.3 Cycle d'appel SIP dans le projet 6
3.4 RTP : transport des flux audio 7
3.5 Qualité de service et limites pratiques 7
3.6 Rôle de SDP 7
3.7 Extraits de code d'implémentation des protocoles 8
3.7.1 Implémentation SIP (construction et sérialisation des messages) . . 8
3.7.2 Implémentation SDP (offre/réponse et parsing) 9
3.7.3 Implémentation RTP (encapsulation et émission des paquets) . . . 10
3.7.4 Implémentation STUN (découverte d'endpoint public UDP) 11
3.7.5 Gestion d'état d'appel SIP (CANCEL vs BYE) 12

4 Diagrammes d'architecture **12**
4.1 Architecture client-serveur 12
4.2 Flux d'un appel et médias RTP 13

5 Capture d'écran de l'application fonctionnelle **13**

6 Tests de connectivité et résultats **17**
6.1 Méthodologie de test 17
6.2 Résultats obtenus 17
6.3 Analyse 17

7 Connexion du système VoIP à Internet : méthodes et choix **18**
7.1 Méthode 1 : serveur SIP public + redirection de ports 18
7.2 Méthode 1 bis : relais RTP côté serveur (implémenté dans ce projet) 18
7.3 Méthode 2 : STUN pour découverte d'endpoint public 19
7.4 Méthode 3 : ICE + TURN (approche robuste recommandée) 19
7.5 Méthode 4 : VPN inter-sites 19
7.6 Choix recommandés selon le contexte 20
7.7 Plan d'évolution proposé pour ce projet 20

8 Conclusion **21**

1 Introduction

La communication en temps réel sur Internet est devenue un composant fondamental des infrastructures numériques modernes. À mesure que les usages collaboratifs se multiplient (télétravail, visioconférence, assistance à distance, e-learning), les exigences en matière de latence, de qualité de service et d'interopérabilité se renforcent. Dans ce contexte, les technologies de Voice over IP (VoIP) occupent une place centrale, car elles permettent d'acheminer la voix via des réseaux IP tout en réduisant les coûts d'exploitation et en augmentant la flexibilité des architectures.

L'objectif de ce projet est de concevoir et implémenter une application VoIP académique complète, reposant sur les protocoles standards SIP (Session Initiation Protocol) pour la signalisation et RTP (Real-time Transport Protocol) pour le transport des flux audio. Le système développé adopte une architecture client-serveur où un serveur SIP joue les rôles de registrar et proxy, tandis que les clients assurent l'enregistrement, l'établissement des appels, la négociation des paramètres média (SDP) et la transmission audio bidirectionnelle.

D'un point de vue pédagogique, ce travail couvre plusieurs dimensions complémentaires :

— la compréhension des protocoles applicatifs de téléphonie sur IP ;

— la modélisation d'une architecture distribuée en environnement UDP ;

— l'implémentation logicielle en Python (socket, threading, gestion d'états) ;

— la validation expérimentale via des tests de connectivité et des scénarios d'appel.

Au-delà de la simple mise en relation de deux correspondants, le projet intègre une interface graphique utilisateur, une gestion de l'historique des appels, des contrôles de session (décrocher, refuser, raccrocher), ainsi qu'un support de traversal NAT via STUN pour améliorer la connectivité dans des contextes réseau réels. L'ensemble constitue une base solide pour l'étude des systèmes de communication multimédia temps réel.

Ce rapport présente, dans une démarche académique structurée, (i) un rappel historique de l'Internet et de WebRTC, (ii) une explication détaillée des protocoles SIP et RTP, (iii) les diagrammes d'architecture du système proposé, (iv) une section dédiée à l'interface fonctionnelle et aux captures d'écran, et (v) les tests réalisés avec leurs résultats. L'objectif final est de démontrer la cohérence entre conception, implémentation et validation, tout en identifiant les limites et perspectives d'évolution (ICE/TURN, sécurité média, scalabilité serveur).

2 Historique : Internet et WebRTC

2.1 Évolution d'Internet

L'histoire d'Internet peut être lue comme la progression d'un réseau de transport de données vers une plateforme de communication universelle. ARPANET (1969) introduit le principe de commutation de paquets : plutôt que de réserver un circuit dédié, les données sont fragmentées et acheminées dynamiquement. Cette rupture technologique constitue la base des communications modernes, y compris la voix sur IP.

Dans les années 1980, la standardisation TCP/IP (adoption officielle sur ARPANET en 1983) permet l'interconnexion de réseaux hétérogènes. Le modèle en couches favorise l'évolution indépendante des services applicatifs : SMTP pour le mail, FTP pour le transfert de fichiers, puis HTTP pour le Web. Avec la démocratisation du World Wide Web dans les années 1990, Internet devient une infrastructure globale orientée grand public.

La décennie 2000 marque le passage d'un Internet principalement asynchrone (pages Web, e-mail) vers un Internet interactif. Les besoins de communication en temps réel augmentent fortement : téléphonie logicielle, centres d'appels IP, visioconférence, travail collaboratif. Or, la voix et la vidéo imposent des contraintes techniques strictes :

— latence faible (idéalement < 150 ms aller simple),

— gigue maîtrisée (variation du délai inter-paquets),

— tolérance limitée aux pertes,

— adaptation aux environnements NAT/pare-feu.

Cette évolution motive l'adoption d'une pile protocolaire spécialisée : SIP pour la signalisation, SDP pour la description média, RTP/RTCP pour le transport et la supervision temps réel.

2.2 De la VoIP classique aux architectures temps réel modernes

Avant WebRTC, les architectures VoIP reposaient sur des clients dédiés (softphones, IP phones) connectés à des serveurs SIP/IMS/PBX. Les défis majeurs étaient déjà présents : négociation des codecs, interopérabilité multi-constructeurs, sécurité de la signalisation, et traversal NAT via STUN/TURN/ALG. Les solutions étaient efficaces mais souvent complexes à déployer dans des environnements hétérogènes.

Progressivement, les mécanismes de qualité de service (priorisation DiffServ, buffers de gigue, adaptation de débit) et de sécurité (SIP over TLS, SRTP) se sont imposés dans les déploiements professionnels. Cette maturation de la VoIP classique a préparé le terrain aux communications temps réel intégrées au navigateur.

2.3 Historique de WebRTC

WebRTC (Web Real-Time Communication), lancé autour de 2011 (Google puis normalisation W3C/IETF), vise à rendre la communication audio/vidéo et data-channel accessible directement depuis le navigateur, sans plugin. L'apport principal est l'intégration native des briques historiquement séparées dans la VoIP : connectivité réseau, sécurité, codecs, et API applicatives.

WebRTC ne remplace pas les principes fondamentaux de la VoIP ; il les encapsule dans un cadre standardisé orienté Web, notamment via :

— **ICE** (Interactive Connectivity Establishment) : découverte et sélection du meilleur chemin réseau,

— **STUN/TURN** : traversal NAT et relais en cas d'échec du pair-à-pair,

— **DTLS-SRTP** : chiffrement et authentification des flux média,

— **API JavaScript** normalisées (`getUserMedia`, `RTCPeerConnection`, `RTCDataChannel`).

Sur le plan architectural, WebRTC réutilise des concepts proches de SIP/RTP (signalisation séparée du média, SDP-like offer/answer, transport RTP sécurisé). La différence est que la signalisation WebRTC est volontairement laissée à l'application (WebSocket, SIP over WebSocket, etc.), alors que SIP définit explicitement les méthodes et transactions.

Dans ce projet, l'approche retenue est SIP/RTP classique (hors navigateur), mais l'intégration de STUN côté client rapproche l'implémentation des contraintes réelles d'Internet public. Cette continuité permet d'articuler clairement les fondements académiques de la VoIP et les pratiques contemporaines issues de WebRTC.

3 Explication des protocoles SIP et RTP

3.1 SIP : signalisation des sessions

SIP (RFC 3261) est un protocole textuel de couche application, inspiré du paradigme requête/réponse. Son rôle est la **signalisation** des sessions : découverte du correspondant, négociation des paramètres multimédia, établissement et terminaison des appels. SIP ne transporte pas directement les données audio ; il pilote la session qui sera ensuite portée par RTP.

Dans une architecture SIP standard, plusieurs entités coopèrent :

— **User Agent Client (UAC)** : émet les requêtes (ex. INVITE),

— **User Agent Server (UAS)** : reçoit et répond,

— **Registrar** : gère l'enregistrement des utilisateurs,

— **Proxy** : relaie/routage des requêtes et réponses.

Le modèle SIP distingue des notions importantes :

— **Transaction** : unité requête/réponses associées,

— **Dialogue** : relation SIP persistante entre deux UA (identifiée par Call-ID + tags),

— **Session** : échange média effectif (RTP).

Méthodes principales utilisées dans le projet :

— `REGISTER` : enregistrement d'un client au serveur SIP,

— `INVITE` : initiation d'appel,

— `ACK` : confirmation d'une réponse finale positive,

— `BYE` : fin d'appel actif,

— `CANCEL` : annulation d'appel en cours d'établissement,

— `OPTIONS` : test de disponibilité/capacité.

extbfCodes de réponse clés : 100 (Trying), 180 (Ringing), 200 (OK), 404 (Not Found), 480 (Temporarily Unavailable), 603 (Decline).

3.2 Structure des messages SIP et champs critiques

Les messages SIP incluent des en-têtes déterminants pour la cohérence protocolaire :

— `Via` : traçabilité du chemin de signalisation ;

— `From`/`To` : identités logique appelant/appelé ;

— `Call-ID` : identifiant global d'appel ;

— `CSeq` : séquencement des requêtes dans un dialogue ;

— `Contact` : adresse de retour directe de l'UA ;

— `Content-Type`/`Content-Length` : gestion du corps SDP.

Dans l'implémentation du projet, la robustesse de ces champs est essentielle : un `Call-ID` stable et un `CSeq` cohérent garantissent la corrélation correcte des réponses, tandis que les informations SDP assurent l'établissement de la session média.

3.3 Cycle d'appel SIP dans le projet

Le scénario nominal suit les étapes suivantes :

1. **REGISTER** de chaque client auprès du serveur ;
2. **INVITE** de l'appelant avec offre SDP ;
3. réponses provisoires **100 Trying** puis **180 Ringing** ;
4. réponse finale **200 OK** avec SDP de l'appelé ;
5. **ACK** de confirmation ;
6. démarrage du flux **RTP** bidirectionnel ;
7. terminaison par **BYE** (appel actif) ou **CANCEL** (appel non établi).

Le projet gère explicitement la distinction `CANCEL` vs `BYE`, ce qui est fondamental pour un comportement utilisateur correct (annuler un appel en sonnerie n'est pas équivalent à terminer un appel actif).

3.4 RTP : transport des flux audio

RTP (RFC 3550) transporte les données audio en temps réel, généralement sur UDP. Le protocole est conçu pour minimiser la latence et tolérer des pertes modérées, contrairement aux mécanismes de retransmission typiques de TCP.

Les champs principaux du header RTP ont un rôle opérationnel direct :

— **Sequence Number** : détection de pertes et réordonnancement,

— **Timestamp** : restitution temporelle des paquets,

— **Payload Type** : identification du codec,

— **SSRC** : identification de la source média.

Le récepteur utilise ces informations avec un *jitter buffer* implicite (ou logique équivalente) pour lisser les variations inter-paquets. La qualité perçue dépend alors d'un compromis entre latence supplémentaire et stabilité audio.

Dans cette implémentation, le flux audio est encodé principalement en G.711 (PCMU/PCMA), avec un framing de 20 ms (160 échantillons à 8 kHz), ce qui correspond à 50 paquets RTP par seconde.

3.5 Qualité de service et limites pratiques

Même en réseau local, les communications temps réel sont sensibles à :

— la **latence** (délai de bout en bout),

— la **gigue** (variation de latence),

— les **pertes de paquets**,

— les **asymétries NAT** dans les contextes Internet.

Le projet intègre STUN pour améliorer l'annonce d'endpoint public RTP. Toutefois, dans les cas de NAT symétrique ou politiques firewall strictes, un relais TURN et une négociation ICE complète deviennent nécessaires pour garantir la connectivité de bout en bout.

3.6 Rôle de SDP

SDP (RFC 4566) encapsule les paramètres techniques de la session média. Dans ce projet, il est transporté dans les messages SIP (INVITE/200 OK) et contient au minimum :

— l'adresse IP et le port RTP (extttc= et `m=audio`),

— les codecs proposés (exttta=rtpmap),

— le mode de transmission (exttta=sendrecv/`recvonly`).

La négociation suit une logique *offer/answer* : l'appelant propose un ensemble de codecs, l'appelé sélectionne un sous-ensemble compatible. Cette étape conditionne directement la compatibilité média et la réussite de l'appel.

3.7 Extraits de code d'implémentation des protocoles

Cette sous-section présente des extraits représentatifs du code réellement utilisé dans le projet pour chaque protocole principal.

3.7.1 Implémentation SIP (construction et sérialisation des messages)

Listing 1 – Sérialisation d'un message SIP (`src/shared/sip_messages.py`)

```
@dataclass
class SIPMessage:
    method: str = ""
    uri: str = ""
    headers: Dict[str, str] = field(default_factory=dict)
    body: str = ""

    def to_bytes(self) -> bytes:
        lines: List[str] = []

        if self.method:
            first_line = f"{self.method} {self.uri} SIP/2.0"
        else:
            status = self.headers.get('Status', '200 OK').strip()
            if status and status.split()[0].isdigit() and len(status.split()) ==
     1:
                status = f"{status} OK"
            first_line = f"SIP/2.0 {status}"

        lines.append(first_line)
        body_bytes = self.body.encode('utf-8') if self.body else b''
        self.headers['Content-Length'] = str(len(body_bytes))

        for header, value in self.headers.items():
            lines.append(f"{header}: {value}")

        lines.append("")
        if self.body:
            lines.append(self.body)

        return "\r\n".join(lines).encode('utf-8')
```

3.7.2 Implémentation SDP (offre/réponse et parsing)

Listing 2 – Création et parsing SDP (src/shared/sip_messages.py)

```
def create_sdp(offer: bool, ip: str, port: int, codecs: List[str] = None) -> str
    :
    if codecs is None:
        codecs = ['PCMU', 'PCMA']

    codec_payload_types = {'PCMU': 0, 'PCMA': 8, 'opus': 123}
    session_id = random.randint(1000, 9999)

    sdp_lines = [
        "v=0",
        f"o=- {session_id} {session_id} IN IP4 {ip}",
        "s=-",
        "c=IN IP4 " + ip,
        "t=0 0",
        "m=audio " + str(port) + " RTP/AVP " + " ".join(str(codec_payload_types.
    get(c, 123)) for c in codecs),
        "a=rtpmap:0 PCMU/8000",
        "a=rtpmap:8 PCMA/8000",
        "a=rtpmap:123 opus/48000/2",
        "a=sendrecv" if offer else "a=recvonly"
    ]
    return "\r\n".join(sdp_lines)

def parse_sdp(sdp_body: str) -> Dict:
    result = {'ip': '', 'port': 0, 'codecs': []}
    for raw_line in sdp_body.split('\r\n'):
        line = raw_line.strip()
        if line.startswith('c=IN IP4 '):
            parts = line.split()
            if len(parts) >= 3:
                result['ip'] = parts[2]
        elif line.startswith('m=audio '):
            parts = line.split()
            if len(parts) >= 2 and parts[1].isdigit():
                result['port'] = int(parts[1])
        elif line.startswith('a=rtpmap:'):
            parts = line.split()
            if len(parts) >= 2:
                encoding = parts[1].split('/')[0].strip()
                if encoding and encoding not in result['codecs']:
```

```
                    result['codecs'].append(encoding)
    return result
```

3.7.3 Implémentation RTP (encapsulation et émission des paquets)

Listing 3 – Paquet RTP et envoi audio (`src/shared/rtp.py`)

```
@dataclass
class RTPPacket:
    version: int = 2
    payload_type: int = 0
    sequence_number: int = 0
    timestamp: int = 0
    ssrc: int = 0
    payload: bytes = b''

    def to_bytes(self) -> bytes:
        first_byte = (self.version << 6)
        second_byte = self.payload_type
        header = struct.pack('!BBHII', first_byte, second_byte,
                             self.sequence_number,
                             self.timestamp & 0xFFFFFFFF,
                             self.ssrc)
        return header + self.payload

class RTPSession:
    def send_audio_frame(self, audio_data: bytes, marker: bool = False) -> bool:
        if not self.socket or not self.remote_addr:
            return False

        self.timestamp = (self.timestamp + 160) & 0xFFFFFFFF
        packet = RTPPacket(
            version=2,
            payload_type=self.payload_type,
            sequence_number=self.sequence_number & 0xFFFF,
            timestamp=self.timestamp,
            ssrc=self.ssrc,
            marker=1 if marker else 0,
            payload=audio_data
        )
        self.socket.sendto(packet.to_bytes(), self.remote_addr)
        self.sequence_number += 1
        return True
```

3.7.4 Implémentation STUN (découverte d'endpoint public UDP)

Listing 4 – Client STUN minimal (`src/shared/stun.py`)

```
MAGIC_COOKIE = 0x2112A442
BINDING_REQUEST = 0x0001
BINDING_SUCCESS_RESPONSE = 0x0101
XOR_MAPPED_ADDRESS = 0x0020

def get_stun_mapped_address(
    udp_socket: socket.socket,
    stun_host: str,
    stun_port: int = 19302,
    timeout: float = 1.5,
) -> Optional[Tuple[str, int]]:
    request, tx_id = _build_binding_request()
    previous_timeout = udp_socket.gettimeout()
    try:
        udp_socket.settimeout(timeout)
        udp_socket.sendto(request, (stun_host, stun_port))
        data, _ = udp_socket.recvfrom(2048)

        msg_type, msg_len, cookie, rx_tx_id = struct.unpack("!HHI12s", data
    [:20])
        if msg_type != BINDING_SUCCESS_RESPONSE:
            return None
        if cookie != MAGIC_COOKIE or rx_tx_id != tx_id:
            return None

        # Parse attributes until XOR-MAPPED-ADDRESS
        # ...
        return parsed
    finally:
        udp_socket.settimeout(previous_timeout)
```

3.7.5 Gestion d'état d'appel SIP (CANCEL vs BYE)

Listing 5 – Terminaison d'appel selon l'état (`src/client/sip_client.py`)

```
def hangup(self):
    if not self.current_call:
        return

    call_status = self.current_call.status
    method = 'CANCEL' if call_status in ('initiating', 'ringing') else 'BYE'

    builder = SIPRequestBuilder(method, f'sip:{self.current_call.remote_user}@{
    self.server_host}')
    builder.call_id = self.current_call.call_id
    builder.from_tag = self.user_id
    builder.add_header('Via', f"SIP/2.0/UDP {self._get_local_ip()};branch={
    generate_via_branch()}")

    request = builder.build()
    self.sip_socket.sendto(request.to_bytes(), (self.server_host, self.
    server_port))
```

4 Diagrammes d'architecture

4.1 Architecture client-serveur

```
Client A (SIP UA)  <------ SIP ------>  Serveur SIP/Proxy  <------ SIP ------>
    Client B (SIP UA)
        |
               |
        +----------------------------- RTP Audio Bidirectionnel
    ----------------------------+
```

FIGURE 1 – Architecture logique client-serveur et chemin des flux.

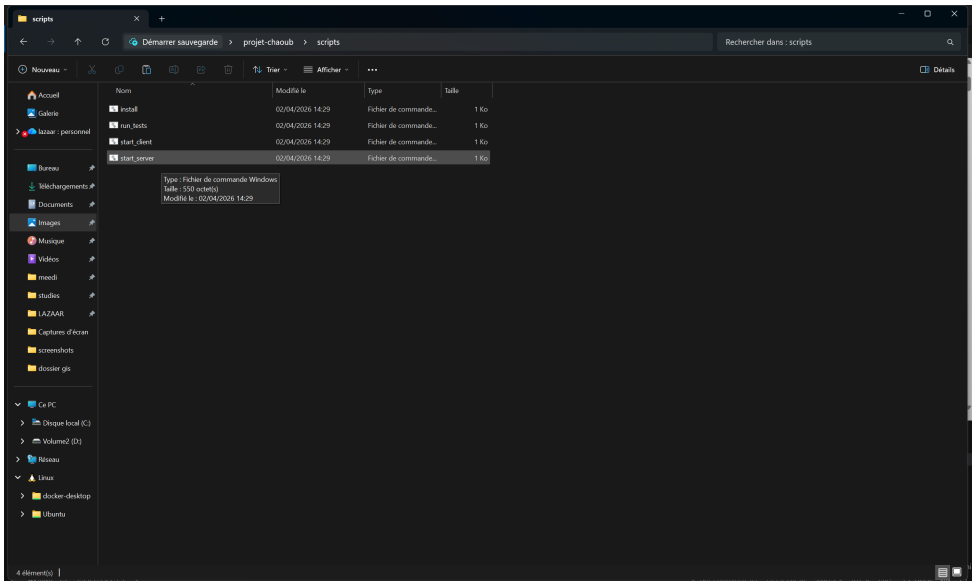
4.2 Flux d'un appel et médias RTP

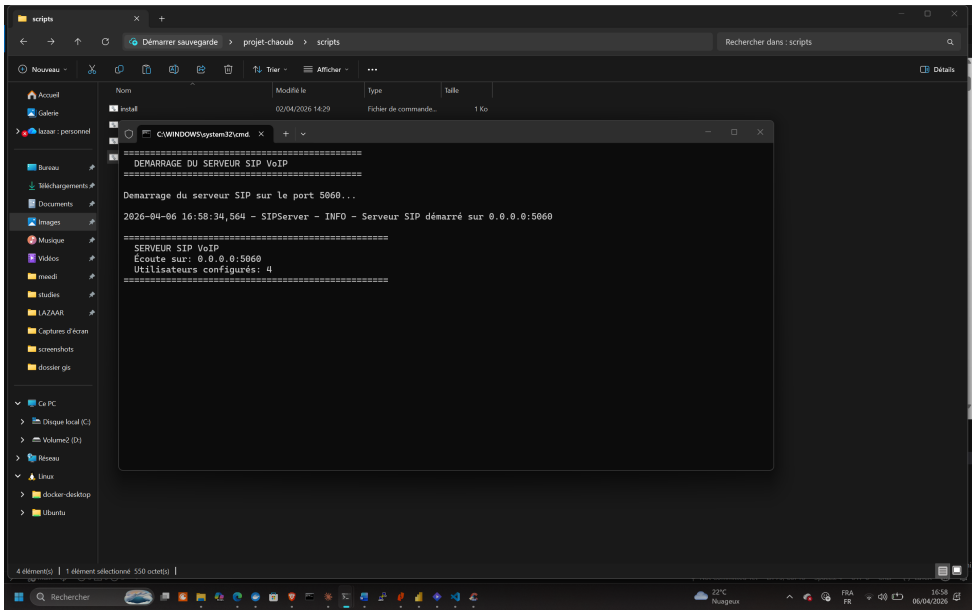
```
Client A                      Serveur SIP                      Client B
   |------ REGISTER ------->|                                  |
   |<------ 200 OK ---------|                                  |
   |------ INVITE --------->|                                  |
   |<----- 100 Trying ------|                                  |
   |                        |------ INVITE ------------------->|
   |                        |<----- 180 Ringing --------------|
   |<----- 180 Ringing -----|                                  |
   |                        |<----- 200 OK -------------------|
   |<------ 200 OK ---------|                                  |
   |------ ACK -----------> |                                  |
   |                        |------ ACK ---------------------->|
   |<======================== RTP STREAM ======================>|
   |------ BYE -----------> |                                  |
   |<------ 200 OK ---------|                                  |
```

FIGURE 2 – Séquence simplifiée SIP + établissement du flux RTP.

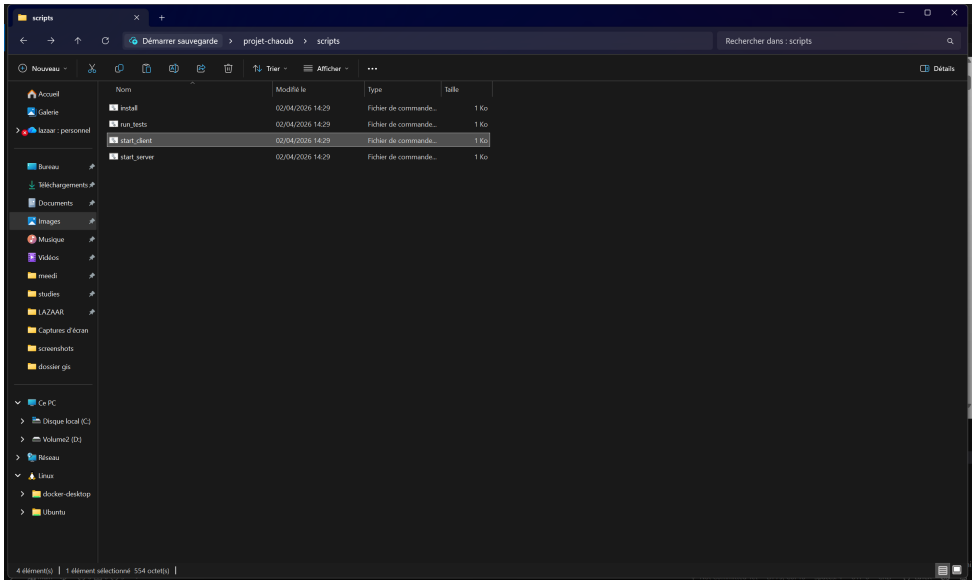
5 Capture d'écran de l'application fonctionnelle

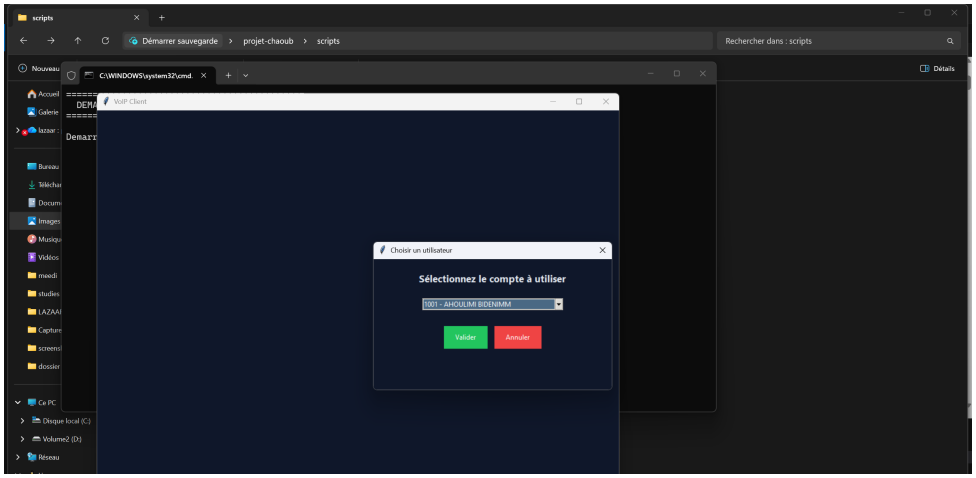
Etape 1 : Lancement du serveur SIP (terminal dédié)

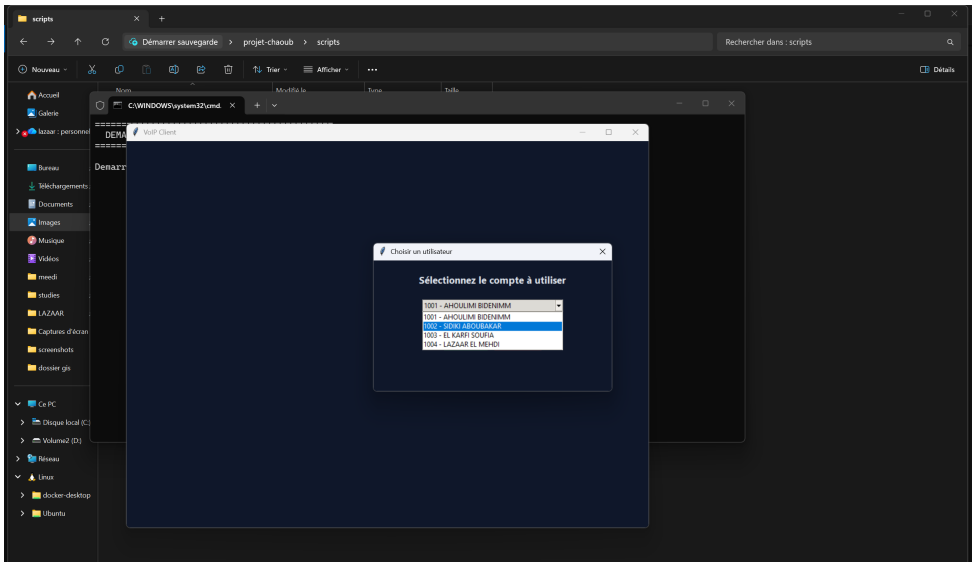


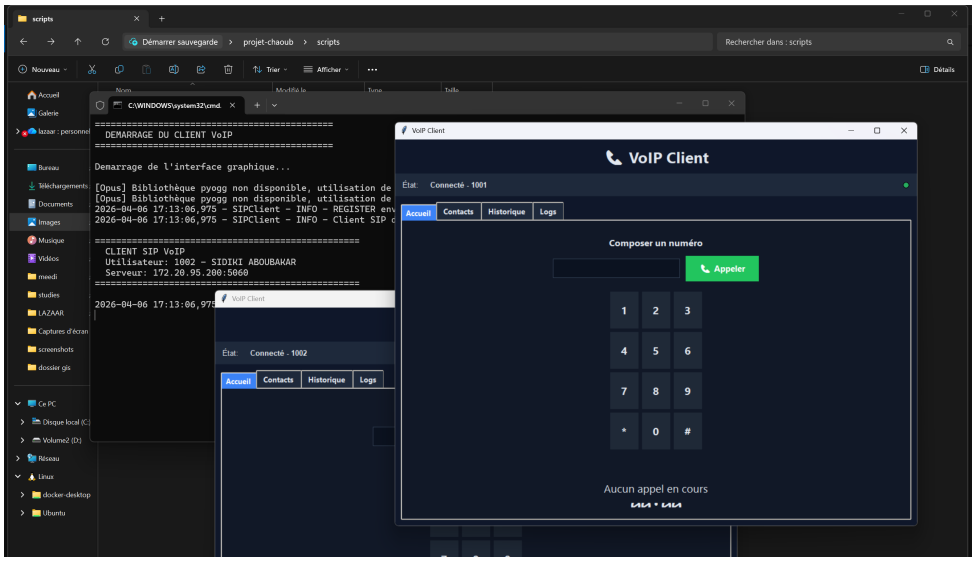


Etape 2 : Lancement des clients et enregistrement

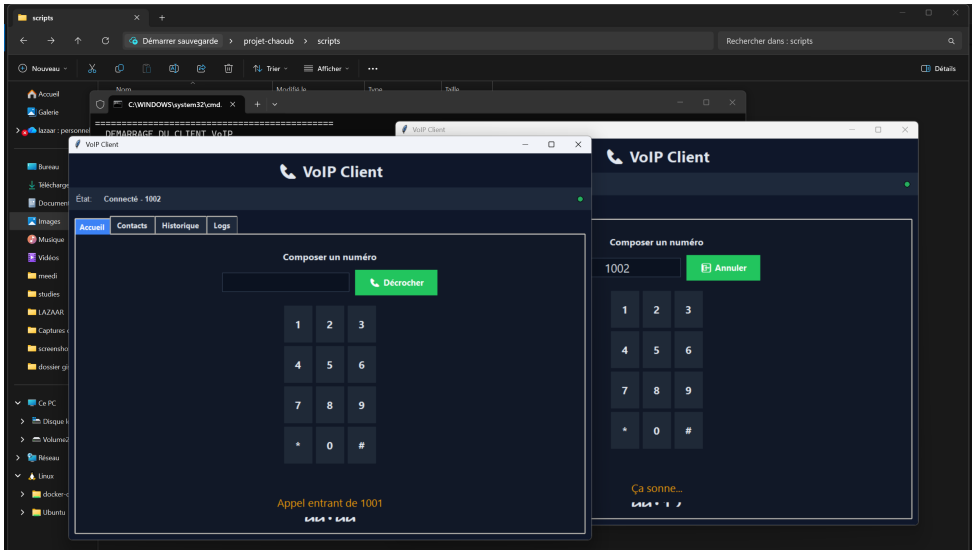


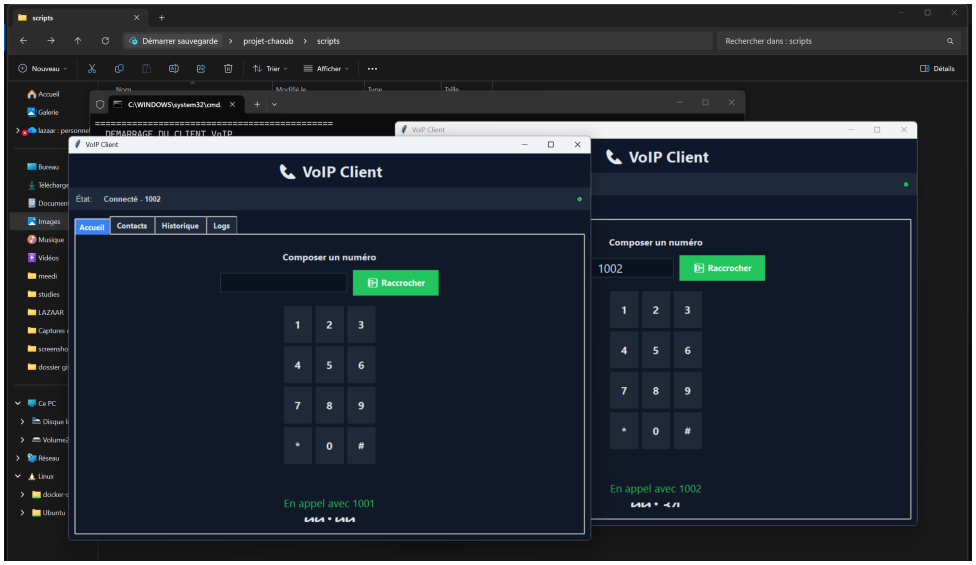



Choisir un utilisateur
Sélectionnez le compte à utiliser
1001 - AHOULIMI BIDENIMM
1002 - SIDIKI ABOUBAKAR
1003 - EL KARFI SOUFIA
1004 - LAZAAR EL MEHDI


DEMARRAGE DU CLIENT VoIP
Demarrage de l'interface graphique...
CLIENT SIP VoIP
Utilisateur: 1002 - SIDIKI ABOUBAKAR
Serveur: 172.20.95.200:5060
VoIP Client
Connecté - 1001
Accueil
Contacts
Historique
Logs
Composer un numéro
Appeler
1 2 3
4 5 6
7 8 9
* 0 #
Aucun appel en cours

Etape 3 : Établissement d'un appel et échange audio





6 Tests de connectivité et résultats

6.1 Méthodologie de test

Les validations ont été conduites en deux étapes :

1. tests unitaires audio/codecs/RTP ;
2. tests de connectivité SIP avec serveur actif.

Scripts utilisés :

— `tests/test_audio.py`

— `tests/test_connectivity.py`

6.2 Résultats obtenus

Campagne	**Résultat**
Tests audio/codecs/RTP	5/5 réussis
Tests connectivité SIP/SDP/RTP	7/7 réussis

TABLE 1 – Synthèse des résultats de test.

Test	**Attendu**	**Statut**
Serveur SIP accessible (OPTIONS)	Réponse 200 OK	OK
REGISTER utilisateur	Enregistrement accepté	OK
INVITE vers utilisateur distant	Réponse valide (Trying/Ringing/OK/Not Found)	OK
Création/Parsing paquet RTP	Entête + payload cohérents	OK
Création/Parsing SDP	IP/Port/Codecs cohérents	OK

TABLE 2 – Extraits des vérifications fonctionnelles.

6.3 Analyse

Les résultats montrent une cohérence entre la logique de signalisation SIP et la couche média RTP. Le système permet l'enregistrement des clients, l'établissement et la terminaison des appels, ainsi que l'échange audio bidirectionnel. L'ajout d'un support STUN améliore la capacité d'annonce d'endpoint public en environnement NAT, tout en conservant un mode local stable pour les tests en réseau local.

7 Connexion du système VoIP à Internet : méthodes et choix

Le passage d'un fonctionnement LAN vers Internet introduit des contraintes supplémentaires : adresses privées non routables, pare-feux d'opérateurs, NAT symétriques et variabilité de latence. Dans ce contexte, la stratégie d'interconnexion doit traiter séparément (i) la signalisation SIP et (ii) le transport média RTP.

7.1 Méthode 1 : serveur SIP public + redirection de ports

La première option consiste à héberger le serveur SIP sur une machine publique (VPS ou cloud) avec IP fixe, puis ouvrir les ports nécessaires : UDP 5060 pour SIP et une plage RTP (par exemple 10000–20000).

Avantages :

— architecture simple à comprendre ;

— déploiement rapide pour des démonstrations ;

— centralisation de la signalisation.

7.2 Méthode 1 bis : relais RTP côté serveur (implémenté dans ce projet)

Une extension concrète a été mise en œuvre dans le code du projet : un mode **relais média côté serveur**. Dans ce mode, le serveur SIP alloue deux ports RTP (un par sens logique), réécrit les SDP de l'offre/réponse, puis relaye les paquets audio entre les deux clients. L'objectif est de contourner les échecs de média direct liés aux NAT restrictifs.

extbfParamètres de configuration associés :

— `server.media_relay_enabled` : active le relais RTP ;

— `server.public_host` : IP publique (ou DNS) annoncée dans le SDP ;

— `server.rtp_port_start / rtp_port_end` : plage de ports relais.

extbfAvantages :

— solution immédiatement opérationnelle pour des scénarios Internet contrôlés ;

— meilleure tolérance aux topologies NAT difficiles que le pair-à-pair pur ;

— cohérence avec une architecture centralisée (serveur SIP déjà présent).

extbfLimites :

— consommation de bande passante serveur (audio relayé) ;

— absence de négociation ICE complète ;

— ne remplace pas TURN/ICE pour une interopérabilité universelle.

Limites :

— le média peut échouer si les deux clients sont derrière des NAT restrictifs ;

— la sécurité reste faible sans TLS/SRTP ;

— exposition directe des ports sur Internet.

7.3 Méthode 2 : STUN pour découverte d'endpoint public

STUN (Session Traversal Utilities for NAT) permet au client de découvrir son adresse et port publics, puis d'annoncer ces paramètres dans SDP. Cette approche est déjà partiellement intégrée dans le projet.

Avantages :

— faible coût d'intégration ;

— améliore la réussite des appels pour NAT de type full-cone ou restricted-cone ;

— conserve un chemin média direct (latence souvent meilleure).

Limites :

— insuffisant face aux NAT symétriques/CGNAT ;

— pas de garantie de connectivité universelle ;

— dépendance à des serveurs STUN externes.

7.4 Méthode 3 : ICE + TURN (approche robuste recommandée)

ICE (Interactive Connectivity Establishment) orchestre plusieurs candidats réseau (hôte local, réflexif STUN, relayé TURN) et choisit automatiquement le meilleur chemin. TURN relaie le média lorsque le pair-à-pair est impossible.

Avantages :

— taux de réussite élevé en environnement Internet réel ;

— robustesse face aux NAT complexes ;

— meilleure expérience utilisateur (moins d'échecs silencieux).

Coûts/contraintes :

— complexité d'implémentation plus importante ;

— besoin d'un serveur TURN dimensionné (bande passante et coût) ;

— logique de sélection/monitoring des candidats à intégrer côté client.

7.5 Méthode 4 : VPN inter-sites

Une alternative pragmatique en contexte académique consiste à placer les deux clients dans un même réseau logique via VPN (WireGuard/OpenVPN). L'application se comporte alors comme en LAN.

Avantages :

— mise en œuvre souvent plus simple qu'ICE/TURN complet ;

— bon niveau de sécurité si le VPN est bien configuré ;

— évite l'exposition directe des ports SIP/RTP au public.

Limites :

— nécessite l'installation/gestion d'une infrastructure VPN ;

— moins adapté à un service grand public ;

— dépendance opérationnelle au tunnel VPN.

7.6 Choix recommandés selon le contexte

oprule **Contexte**	**Choix recommandé**	**Justification**
Prototype local/LAN	SIP serveur local + RTP direct	Simplicité, faible latence, débogage rapide
Démo Internet contrôlée	Serveur SIP public + relais RTP + STUN	Déploiement réalisable rapidement sur 2 PC distants
Production multi-opérateurs	SIP public + ICE + TURN + TLS/SRTP	Meilleure robustesse et sécurité
Usage académique distant	VPN + serveur SIP interne	Déploiement stable sans exposition publique forte

TABLE 3 – Comparaison des stratégies de connexion Internet pour la solution VoIP.

7.7 Plan d'évolution proposé pour ce projet

Pour rendre l'application durablement opérationnelle entre deux PC distants sur Internet, la trajectoire la plus pertinente devient :

1. conserver le mode relais RTP serveur comme base fonctionnelle initiale ;
2. déployer le serveur SIP sur un hôte public avec IP fixe (ou DNS) ;
3. sécuriser la signalisation (SIP over TLS) et le média (SRTP) ;
4. intégrer un mécanisme ICE/TURN pour améliorer l'interopérabilité globale ;
5. instrumenter des métriques de qualité (latence, gigue, pertes) pour pilotage adaptatif.

Cette progression conserve la base actuelle du projet tout en améliorant progressivement l'interopérabilité Internet, la sécurité et la fiabilité des appels.

8 Conclusion

Ce projet démontre la réalisation complète d'une chaîne VoIP académique basée sur des standards ouverts. L'architecture mise en place confirme la séparation claire entre signalisation (SIP/SDP) et média (RTP), ainsi que la pertinence d'un serveur SIP léger pour l'orchestration des appels.

Sur le plan pédagogique, les objectifs sont atteints : compréhension des protocoles, implémentation opérationnelle, tests reproductibles, et interface utilisateur exploitable. Les perspectives d'amélioration incluent :

— intégration ICE/TURN complète pour traversée NAT avancée,

— sécurisation média (SRTP) et signalisation (TLS),

— enrichissement des métriques de qualité (gigue, perte, MOS estimé),

— extension vers des conférences multi-participants.

Références

[1] J. Rosenberg et al., *SIP : Session Initiation Protocol*, RFC 3261, IETF, 2002.

[2] H. Schulzrinne et al., *RTP : A Transport Protocol for Real-Time Applications*, RFC 3550, IETF, 2003.

[3] M. Handley et al., *SDP : Session Description Protocol*, RFC 4566, IETF, 2006.

[4] A. Bergkvist et al., *WebRTC 1.0 : Real-time Communication Between Browsers*, W3C Recommendation.